

Профилирование CPU: BenchmarkDotNet и dotnet-trace

C# Developer Advanced



Проверить, идет ли запись

Меня хорошо видно & слышно?





Орехов Антон

Программист интеграции

- Опыт разработки в Production > 20 лет C#, python
- Fullstack, микросервисы

 @Antoni_www

 mailto-Anton@yandex.ru

Маршрут вебинара

Знакомство

Основы профилирования CPU

BenchmarkDotNet

dotnet-trace

Итоги



Цели вебинара

К концу занятия вы сможете

1. Измерять производительность методов с помощью BenchmarkDotNet для принятия обоснованных решений по оптимизации
2. Использовать dotnet-trace для сбора трассировок и анализа производительности приложения в целом
3. Находить "горячие" точки (hot paths) в коде, которые потребляют больше всего процессорного времени



Смысл

Это нужно уметь чтобы:

1. Находить причину потери производительности в коде

2. Обоснованно делать решения об оптимизациях ПО



Профилирование CPU в .NET

Профилирование CPU в .NET

Алгоритм:

1. Собрать метрики
2. Понять причину
3. Отрефакторить код
4. Перепроверить

Инструменты:

- **BenchmarkDotNet** — микроизмерения *методов* (скорость/аллокации/статистика)
- **dotnet-trace** — системная трассировка *процесса* (EventPipe, hot paths)



Почему нельзя оптимизировать «на глаз»

- Интуитивный подход часто ведет разработчиков мимо по-настоящему «горячих» мест
- Локальная оптимизация и общее ускорение системы – это не одно и то же
- Решения об оптимизации нужно принимать только по измерениям и фактам, чтобы иметь возможность валидировать результат

Виды измерений: ручные, микробенчи, трассировки и счётчики

Вид измерения	Для чего	Плюсы	Минусы
Ручные замеры	Экспресс-оценка	Просто, без зависимостей	Шум, нет статистики
Микробенчмарки (BDN)	Сравнить варианты метода	Прогрев, повторения, статистика, аллокации	Не знает «почему» и контекст системы
Трасса (dotnet-trace)	Найти горячие пути в процессе	Реальная нагрузка, flame graph, стек вызовов	Сэмплирование, не для тонких сравнений
Счётчики (dotnet-counters)	Быстрые показатели (CPU%, GC)	Моментальный «пульс»	Мало деталей

Шум и правила чистого эксперимента

Ручной замер таймером – не профилирование

Источники шума:

1. первый запуск (JIT),
2. сборщик мусора (GC),
3. фоновые процессы,
4. отладчик/Debug,
5. энергосбережение ОС и CPU.

Правила чистоты эксперимента:

1. Release, x64, без отладчика;
2. прогрев перед замером;
3. несколько итераций.

BenchmarkDotNet автоматизирует эти правила



JIT, Tiered Compilation, R2R, PGO

JIT (Just-In-Time)

- Компилирует IL в машинный код **при первом вызове метода**.
- Первый запуск обычно **медленнее**, последующие — быстрее.

Tiered Compilation

- Двухступенчатая JIT-компиляция:
 - Tier0** — быстро и грубо (быстрый старт),
 - Tier1** — позже, **лучше оптимизированно** для «горячих» методов.
- В процессе работы скорость **может меняться**.

R2R (ReadyToRun)

- **Предкомпиляция заранее** при публикации.
- Ускоряет **старт**, но «пиковая» скорость может быть ниже, чем у хорошо разогретого JIT.
- Не весь код предкомпилируется: часть всё равно JIT-ится (например, некоторые обобщения).

PGO (Profile-Guided Optimization)

- **Оптимизация по профилю**: среда собирает, **что часто выполняется**, и подстраивает код (инлайнинг, раскладка веток и т.д.).
- Бывает **динамическая** (во время работы) и **статическая** (на этапе сборки/публикации).

CPU-bound vs I/O-bound: когда CPU-профилирование помогает, а когда — нет

CPU-bound (упираемся в процессор)

- Время уходит на **вычисления**
- Высокая загрузка CPU
- Помогают: **BenchmarkDotNet + cpu-sampling**

I/O-bound (упираемся в ожидание)

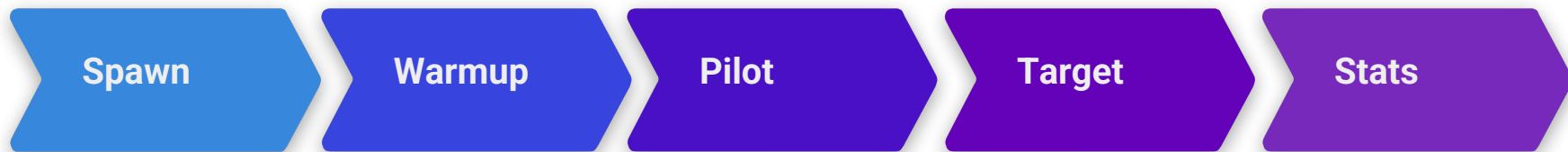
- Время уходит на **ожидание** (сеть/диск/БД)
- CPU часто простаивает
- Помогают: **трассы + счётчики** (latency, очереди), профили I/O

Ключевые различия

- **CPU-время** (сколько реально считал процессор) $\neq$ **время по часам**
- На flame graph:
 - CPU-bound — широкие полосы «нашего» кода
 - I/O-bound — много ожиданий/разрывов

BenchmarkDotNet

BenchmarkDotNet: архитектура и фазы



Бенчи идут не внутри вашей IDE-процесса, а в чистом процессе. Так меньше влияния отладчика и расширений.

Первые прогоны нужны, чтобы прошла JIT-компиляция и «успокоились» кэши. После прогрева измерения стабильнее.

Инструмент подбирает, сколько вызовов делать в одной итерации, чтобы сигнал был достаточно длинным и измеряемым.

Выполняются десятки и сотни итераций, чтобы получить распределение — не одно число, а набор наблюдений.

Считаются среднее/медиана/разброс, выявляются выбросы, формируются отчёты.

BenchmarkDotNet: Jobs и Runtime (как честно сравнивать .NET 9 / .NET Framework)

- **Job** = набор настроек запуска: Runtime, JIT, платформа (x64/x86), GC, env-переменные
- Сравниваем **реализации** — 1 Job; сравниваем **среды** — несколько Jobs
- Частый кейс: .NET 8 (CoreCLR) vs .NET Framework 4.7.2/4.6.1 (CLR)
- Честное сравнение: **одни и те же данные, один и тот же код, разные Jobs**

```
// .NET 8, x64, Server GC
var net8:Job = Job.Default
    .WithId(".NET 8 / ServerGC")
    .WithRuntime(CoreRuntime.Core80)
    .WithPlatform(Platform.X64)
    .WithJit(Jit.RyuJit)
    .WithGcServer(true);

// .NET 9, x64, Workstation GC
var net9:Job = Job.Default
    .WithId(".NET 9 / WorkstationGC")
    .WithRuntime(CoreRuntime.Core90)
    .WithPlatform(Platform.X64)
    .WithJit(Jit.RyuJit)
    .WithGcServer(false);

AddJob(net8);
AddJob(net9);
```

Diagnosers в BenchmarkDotNet: что собирать и как читать

MemoryDiagnoser

- Показывает **аллокации** (Allocated) и сборки GC (Gen0/1/2)
- Помогает ловить «быстро, но мусорит»

ThreadingDiagnoser

- Показывает **блокировки/контеншн**, пулы потоков
- Уместно при многопоточности/параллелизме

DisassemblyDiagnoser

- Даёт **IL и машинный код** (JIT ASM)
- Видно: инлайнинг, лишние вызовы, ветвления

Когда включать

- По умолчанию — **Memory**
- При подозрении на синхронизацию — **Threading**
- Для глубокой разборки — **Disassembly** (точно)

```
[MemoryDiagnoser, RankColumn, Orderer(SummaryOrderPolicy.FastestToSlowest)]  
Ссылка 1  
public class AntiPatternsDemo  
{  
    private readonly Consumer _consumer = new();  
  
    [Params(1_000, 100_000)] public int N;  
    private int[] _data = null!;
```

```
BenchmarkDotNet v0.15.4, windows 11 (10.0.26100.6057/24H2/2824/update/HudsonValley)  
AMD Ryzen 5 3600 3.60GHz, 1 CPU, 12 logical and 6 physical cores  
.NET SDK 10.0.200  
[Host] : .NET 8.0.25 (8.0.25, 8.0.2526.11203), X64 RyuJIT x86-64-v3 [AttachedDebugger]  
.NET 9 / WorkstationGC : .NET 9.0.14 (9.0.14, 9.0.1426.11910), X64 RyuJIT x86-64-v3  
.NET 8 / ServerGC : .NET 8.0.25 (8.0.25, 8.0.2526.11203), X64 RyuJIT x86-64-v3  
Jit=RyuJit Platform=X64
```

Method	Job	Runtime	Server	Mean	Error	StdDev	Median	Min	Max	Rank	Rank	Allocated
ForLoop	.NET 9 / WorkstationGC	.NET 9.0	False	387.5 ns	4.35 ns	4.07 ns	388.5 ns	381.2 ns	392.0 ns	1	*	-
ForLoop	.NET 8 / ServerGC	.NET 8.0	True	393.7 ns	6.90 ns	6.46 ns	390.0 ns	383.9 ns	402.6 ns	1	*	-

Конфигурация и анти-паттерны: как не обмануть себя в BenchmarkDotNet

Анти-паттерны

- **Dead code elimination**: результат нигде не используется
→ код «выкинут»
- **Слишком короткие бенчи**: шум таймера сильнее сигнала
- **Неравные условия**: разный инлайнинг/JIT/GC/платформа
- **Генерация данных внутри замера**
- **I/O/Sleep в микробенче** (это уже не микробенч)

Рецепты

- Возвращать результат **или** Consumer/Blackhole
- Увеличить сигнал: InvocationCount, UnrollFactor, IterationTime
- Зафиксировать среду: Job, NoInlining (точечно), одинаковые данные
- GlobalSetup/IterationSetup для подготовки данных
- Для I/O — не BDN, а трассировки/счётчики

```
//результат не используется – оптимизатор может выбросить работу
[Benchmark(Baseline = true)]
% Оптимизировать Ссылка: 0
public void Bad_Sum()
{
    int s = 0;
    for (int i = 0; i < _data.Length; i++) s += _data[i];
    // ничего не возвращаем и не потребляем
}

// потребляем результат, чтобы запретить выброс
[Benchmark]
% Оптимизировать Ссылка: 0
public void Good_Sum_Consume()
{
    int s = 0;
    for (int i = 0; i < _data.Length; i++) s += _data[i];
    _consumer.Consume(s); // или return s;
}
```

Параметризация, Baseline и чтение отчётов

Параметры ввода

- [Params(...)] и [ParamsSource] — масштаб и вариации данных
- [GlobalSetup] — готовим данные заранее

Сравнение с опорой

- [Benchmark(Baseline = true)] — «нулевая» точка
- **Ratio** — во сколько раз быстрее/медленнее при одинаковом Job и Params

Как читать отчёт

- **Median** — устойчивая оценка; **Mean** — среднее
- **StdDev / Error / CI** — разброс и уверенность
- **Allocated / Gen0+** — цена по памяти
- **Rank** — ранжирование (быстрее → лучше)

```
[Params(1_000, 10_000, 100_000)]  
public int N;  
  
[Params("hit", "miss", "mixed")]  
public string Mode = null!;
```

```
[GlobalSetup]  
Ссылка 0  
public void Setup() => _data = Enumerable.Range(1, N).ToArray();  
  
//результат не используется - оптимизатор может выбросить работу  
[Benchmark(Baseline = true)]  
% Оптимизировать Ссылка 0  
public void Bad_Sum()  
{  
    int s = 0;  
    for (int i = 0; i < _data.Length; i++) s += _data[i];  
    // ничего не возвращаем и не потребляем  
}  
  
// потребляем результат, чтобы запретить выброс  
[Benchmark]  
% Оптимизировать Ссылка 0  
public void Good_Sum_Consume()  
{  
    int s = 0;  
    for (int i = 0; i < _data.Length; i++) s += _data[i];  
    _consumer.Consume(s); // или return s;  
}  
  
[MethodImpl(MethodImplOptions.NoInlining)]  
Ссылка 1  
private static int Add(int a, int b) => a + b;
```

Зачем смотреть разбор кода и что в нём искать

- **Задача:** понять **почему** один способ быстрее другого

- **Ищем три вещи:**

1. Вызов маленького метода **встроен** в цикл или вызывается отдельно
2. Есть ли **лишние создания объектов**
3. Есть ли **лишние проверки** внутри цикла

- **Как включить в BDN:** DisassemblyDiagnoser
(включаем **точечно**, отчёты тяжёлые)

Ограничения BenchmarkDotNet

BDN работает в лабораторных условиях

- Измеряет **отдельный метод** в **чистых условиях**
- Не видит взаимодействий: сеть/диск/БД, очереди, пулы, логи

Где BDN вводит в заблуждение

- **Нереалистичные данные** (слишком простые/равномерные)
- **Горячие кэши** и «разогретый» код → лучше, чем в реальности
- **Параллелизм/блокировки** почти не отражены
- Побочные эффекты: **логирование, исключения, ретраи**

dotnet-trace

dotnet-trace: что это и как работает

• **Задача:** подключиться к **уже запущенному** .NET-процессу и записать **трассировку** — файл с событиями выполнения (.nettrace)

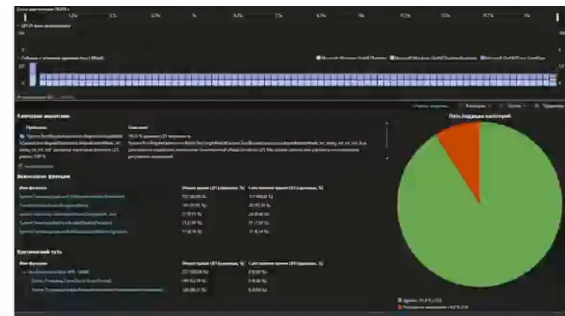
• **Что видим в файле:**

- «снимки стека» по CPU — какие методы реально занимали процессор
- события сборщика мусора (GC) и компилятора «на лету» (JIT)
- переключения потоков и их вклад

• **Где смотреть:** PerfView или Visual Studio (Diagnostic Tools)

• **Готовые профили:**

- **cpu-sampling(dotnet-common)** — «горячие» функции (главный профиль для CPU)
- **gc** (gc-collect / gc-verbose) — паузы и частота сборок мусора



```
PS C:\Users\Dmitry> dotnet-trace ps
3816  DevHub      ity.and6\inet8_0-windows8_0\DevHub.exe  e\BC6395D2-A208-45E7-9584-EB3DADBFCBBA
27684  Otus.Benchmark.Test6  a\usecase\inet8_0\Otus.Benchmark.Test6.exe  Otus.Benchmark.Test6.exe" cpu-slow 60
37520  VBCSCompiler  .0_280\Roslyn\bin\core\VBCSCompiler.exe  a6NQtqPhP9Gu09Xhs_dt51bJTomRyRQLTEdc"
1924   auapi        vated process - cannot determine path]
5156   dotnet       C:\Program Files\dotnet\dotnet.exe         net.exe" run -c Release -- cpu-slow 60
28560  kpm          vated process - cannot determine path]
5520   kpm_isolation 25.1\kpm_isolation.exe                 hannel 5688:5684 -traces_enabled False
15544  pwsh         C:\Program Files\PowerShell\7\pwsh.exe     :\Program Files\PowerShell\7\pwsh.exe"
21364  pwsh         C:\Program Files\PowerShell\7\pwsh.exe     :\Program Files\PowerShell\7\pwsh.exe"
23168  pwsh         C:\Program Files\PowerShell\7\pwsh.exe     :\Program Files\PowerShell\7\pwsh.exe"

PS C:\Users\Dmitry> dotnet-trace collect -p 27684 --profile dotnet-common -o cpu-slow.nettrace

Provider Name      Keywords      Level      Enabled By
-----
Microsoft-Windows-DotNETRuntime  0x0000001000000010  Informational(4)  --profile

Process           : C:\Users\Dmitry\Downloads\Otus.Benchmark-290269-9f3855\Otus.Benchmark\Otus.Benchmark.Test6\
net8_0\Otus.Benchmark.Test6.exe
[00:00:00:10] :Recording trace 118,784 (KB)trace
Press <Enter> or <Ctrl+C> to exit...
```

Профили dotnet-trace: dotnet-common и gc

cpu-common (по умолчанию для CPU)

- Частые «снимки стека» → видно, **какие методы реально грузят процессор**
- Основной инструмент, чтобы найти **горячие точки** (hot paths)
- Небольшая нагрузка, компактный файл

gc-collect / gc-verbose (для памяти)

- **gc-collect**: когда и как часто шёл сборщик мусора (короткий отчёт)
- **gc-verbose**: подробности пауз и объёмов, кто «мусорит» (подробнее, файл больше)
- Нужен, когда видим **скачки времени** из-за памяти

Keyword string alias

gc

gchandle

assemblyloader

loader

jit

ngen

startenumeration

endenumeration

security

appdomainresourcemangement

jittracing

interop

contention

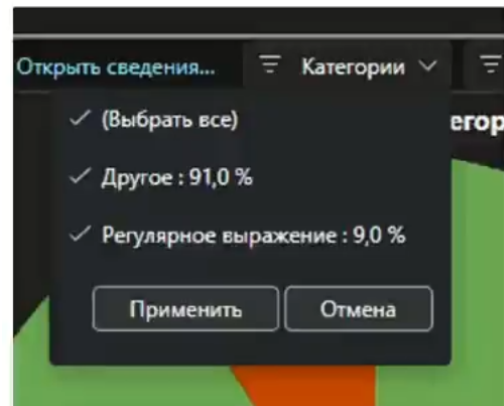
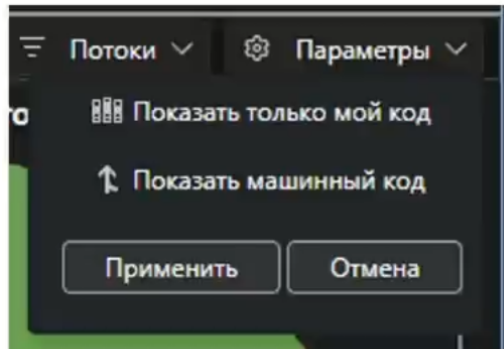
Как быстро находить «горячие точки» в файле трассировки

Быстрые фильтры

- Спрятать «мелочь»: порог **Fold %** (PerfView) или скрыть «меньше 1%»
- Показать «только мой код» (Just My Code / фильтр по namespace)
- Для async-путей: включить «склеивание» асинхронных кадров (VS/PerfView)

Шаблоны, на которые смотреть

- **Широкая полка** одной функции → настоящий «горячий» цикл
- Каскад Json/Regex/String → тяжёлый парсинг/форматирование
- Частые узлы GC рядом с нашим кодом → много аллокаций
- Много «внешнего кода» и тонкие наши полосы → вероятно, I/O-ожидание



Когда dotnet-trace не хватает: что ещё использовать и в каких случаях

Когда «узкое место» вне кода .NET

- Активный диск/сеть, драйверы, конкуренция с другими процессами
→ **Windows Performance Recorder/Analyzer (WPR/WPA)**: системная запись, видно CPU/диск/сеть/контекстные переключения

Нужна очень подробная картина внутри .NET Framework 4.x

- dotnet-trace не работает с .NET Framework
→ **PerfView (ETW-сессия)** или **VS Profiler (CPU Usage / Instrumentation)**

Подозрение на «нативные» библиотеки

- Сжатие, криптография, C/C++ DLL
→ **WPR/WPA + символы (PDB/Source Link)**, иначе имена будут «без названий»

Очень короткий сценарий (старт/инициализация)

- Процесс быстро завершается, не успеваем подключиться
→ Запуск через dotnet-trace collect -- <ваша_команда> (сбор «с нуля»)

Вопросы



если есть вопросы



если вопросов нет



Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

